- **Simple functions. Functions without parameters.**

Example 1:

```
// Example 1
// demonstrates simple function
#include <iostream>
#include <iomanip>
using namespace std;

void starline();                        // starline function prototype (declaration)

int main()
{
       starline();                      // starline function call
       cout << setiosflags(ios::left) << setw(15) << "Data type" << "Range" << endl;
       starline();                      // starline function call
       cout << setiosflags(ios::left)
            << setw(15) << "char" << "-128...127" << endl
            << setw(15) << "short" << "-32,768...32,767" << endl
            << setw(15) << "int" << "platform dependent" << endl
            << setw(15) << "long" << "-2,147,483,648...2,147,483,647" << endl;
       starline();                      // starline function call
       cout << endl;
       return 0;
}

//-------------------------------------------------------
// starline function definition
// 50 times outputs '*'
void starline()                         // starline function header
{                                       // starline function body begins
       for(int j = 0; j < 50; j++)
              cout << '*';
       cout << endl;
}
```

Example 2:

```
// Example 2
// demonstrates simple function definition without specifying prototype
#include <iostream>
#include <iomanip>
using namespace std;

//-------------------------------------------------------
// starline function definition
void starline()                         // starline function header
{                                       // starline function body begins
       for(int j = 0; j < 50; j++)
              cout << '*';
       cout << endl;
}

int main()
{
       starline();                      // starline function call
       cout << setiosflags(ios::left) << setw(15) << "Data type" << "Range" << endl;
       starline();                      // starline function call
       cout << setiosflags(ios::left)
            << setw(15) << "char" << "-128...127" << endl
            << setw(15) << "short" << "-32,768...32,767" << endl
            << setw(15) << "int" << "platform dependent" << endl
            << setw(15) << "long" << "-2,147,483,648...2,147,483,647" << endl;
       starline();                      // starline function call
       cout << endl;
       return 0;
}
```

- **Passing constants to a function**

Example 3:

```
// Example 3
// passing constant values as arguments to function
#include <iostream>
#include <iomanip>
using namespace std;

void charline(char, int);              // charline function prototype (declaration)

int main()
{
       charline('-', 45);              // charline function call
       cout << setiosflags(ios::left) << setw(15) << "Data type" << "Range" << endl;
       charline('=', 30);              // charline function call
       cout << setiosflags(ios::left)
            << setw(15) << "char" << "-128...127" << endl
            << setw(15) << "short" << "-32,768...32,767" << endl
            << setw(15) << "int" << "platform dependent" << endl
            << setw(15) << "long" << "-2,147,483,648...2,147,483,647" << endl;
       charline('-', 45);              // charline function call
       cout << endl;
       return 0;
}

//-------------------------------------------------------
// charline function definition
void charline(char ch, int n)          // charline function header
{                                      // charline function body begins
       for (int j = 0; j < n; j++)
              cout << ch;
       cout << endl;
}
```

- **Passing variables to a function**

Example 4:

```
// Example 4
// passing variables as arguments to function
#include <iostream>
using namespace std;

void charline(char, int);              // charline function prototype (declaration)

int main()
{
       char symbol;
       int count;
       cout << "Enter symbol: ";
       cin >> symbol;
       cout << "Enter symbol count: ";
       cin >> count;
       charline(symbol, count);
       return 0;
}

//-------------------------------------------------------
// charline function definition
void charline(char ch, int n)          // charline function header
{                                      // charline function body begins
       for (int j = 0; j < n; j++)
              cout << ch;
       cout << endl;
}
```

- **Returning a value from a function**

Example 5:

```
// Example 5
// demonstrates how function can return a value
#include <iostream>
using namespace std;

float kmhtoms(float);                   // kmhtoms function prototype (declaration)

int main()
{
       float kmh, ms;
       cout << "\nEnter speed in km/h: ";
       cin >> kmh;
       ms = kmhtoms(kmh);
       cout << "Speed in m/s: " << ms << endl << endl;
       return 0;
}

//-------------------------------------------------------
// kmhtoms function definition
// converts speed value in km/h to speed value in m/s and returns it
float kmhtoms(float fkmh)
{
       float fms = fkmh * 1000 / 3600;  // or simply float fms = fkmh / 3.6
       return fms;
}
```

Example 6:

```
// Example 6
// demonstrates how function can return a value (more compact version)
#include <iostream>
using namespace std;

float kmhtoms(float);                   // kmhtoms function prototype (declaration)

int main()
{
       float kmh;
       cout << "\nEnter speed in km/h: ";
       cin >> kmh;
       cout << "Speed in m/s: " << kmhtoms(kmh) << endl << endl;
       return 0;
}

//-------------------------------------------------------
// kmhtoms function definition
// converts speed value in km/h to speed value in m/s and returns it
float kmhtoms(float fkmh)
{
       return fkmh * 1000 / 3600;       // or simply return fkmh / 3.6
}
```

- **Passing variable *by pointer***

Example 7:

```
// Example 7
// passing arguments by pointer
#include <iostream>
using namespace std;

void feetToMetres(double*);            // feetToMetres function prototype (declaration)

int main()
{
       double var = 15.0;                      // variable var value is 15 feet
       cout << "var = " << var << " feet" << endl;

       feetToMetres(&var);                           // convert feet to metres
       cout << "var = " << var << " metres" << endl << endl;

       return 0;
}

//-------------------------------------------------------
// feetToMetres function definition
// converts feets to metres
void feetToMetres(double* ptrv)
{
       *ptrv = *ptrv * 0.3048;                 // *ptrv is the same as var in main function
}
```

- **Passing variable *by reference* and „returning” a value through parameter**

Example 8:

```
// Example 8
// passing arguments to function by reference
#include <iostream>
using namespace std;

void feetToMetres(double&);            // feetToMetres function prototype (declaration)

int main()
{
       double var = 15.0;                        // variable var value is 15 feet
       cout << "var = " << var << " feet" << endl;

       feetToMetres(var);                                // convert feet to metres
       cout << "var = " << var << " metres" << endl << endl;

       return 0;
}

//-------------------------------------------------------
// feetToMetres function definition
// converts feets to metres
void feetToMetres(double& v)
{
       v = v * 0.3048;                           // v is the same as var in main function
}
```

Example 9:

```
// Example 9
// passing arguments to function by reference
#include <iostream>
using namespace std;

void getIntAndFracPart(float, int&, float&);    // getIntAndFracPart function prototype
                                                // (declaration)

int main()
{
       float number, fracpart;
       int intpart;
       do {
              cout << "\nEnter real number: ";
              cin >> number;
              getIntAndFracPart (number, intpart, fracpart); // defining integer and
                                                             // fractional part of the number
              cout << "Integer part is " << intpart
                   << ", fractional part is " << fracpart
                   << endl;
       } while (number != 0.0);                   // exit from the loop if 0.0 entered
       cout << endl;
       return 0;
}

//-------------------------------------------------------
// getIntAndFracPart function definition
// calculates integer and fractional part of a real number
void getIntAndFracPart(float n, int& intp, float& fracp)
{
       intp = static_cast<int>(n);                // type casting from float to int
       fracp = n - intp;                          // substracting integer part
}
```

- **Passing array to a function**

Example 10:

```
// Example 10
// passing array as argument to function
#include <iostream>
#include <time.h>
using namespace std;

void generate_array(int[],int);                // generate_array function prototype
void display_array(int[],int);                 // display_array function prototype

int main()
{
       const int n = 5;
       int arr[n];    // static array
       //int n = 5;
       //int* arr = new int[n];                // dynamic array
       generate_array(arr,n);                  // generate_array function call
       display_array(arr,n);                   // display_array function call

       //delete [] arr;                        // needed if array was created dynamically
       cout << endl << endl;

       return 0;
}
```

```
//-------------------------------------------------------
// generate_array function definition
// generates array of random numbers
void generate_array(int mas[], int size)
{
	srand(time(NULL));
	for(int i = 0; i < size; i++)
		mas[i] = rand()%1000;
}


//-------------------------------------------------------
// display_array function definition
// outputs array
void display_array(int mas[], int size)
{
	for (int i = 0; i < size; i++)
		cout << mas[i] << " ";
}
```

- **Passing array *by pointer***

Example 11:

```
// Example 11
// passing array as argument to function (by pointer)
#include <iostream>
#include <time.h>
using namespace std;

void generate_array(int*,int);		// generate_array function prototype
void display_array(int*,int);		// display_array function prototype

int main()
{
	const int n = 5;
	int arr[n];	// static array
	//int n = 5;
	//int* arr = new int[n];		// dynamic array
	generate_array(arr,n);		// generate_array function call
	display_array(arr,n);		// display_array function call

	//delete [] arr;		// needed if array was created dynamically

	return 0;
}

//-------------------------------------------------------
// generate_array function definition
// generates array
void generate_array(int* mas, int size)
{
	srand(time(NULL));
	for (int i = 0; i < size; i++)
		mas[i] = rand()%1000;
		//*(mas+i) = rand()%1000;

}

//-------------------------------------------------------
// display_array unction definition
// outputs array
void display_array(int* mas, int size)
{
	for (int i = 0; i < size; i++)
		cout << mas[i] << " ";
		//cout << *(mas+i) << " ";
}
```

Example 12:

```
// Example 12
// passing array as argument to function (by pointer)
#include <iostream>
using namespace std;

const int MAX = 5;                      // element count in array as global variable

void feetToMetres(double*);             // feetToMetres function prototype

int main()
{
       double varray [MAX] = { 15.0, 34.6, 75.9, 23.7, 55.1 };

       feetToMetres(varray);            // convert all array elements from feet to metres

       // output the result
       for (int j = 0; j < MAX; j++)
              cout << "varray [ " << j << " ] = " << varray [ j ] << " metres" << endl;

       cout << endl;
       return 0;
}

//-------------------------------------------------------
// feetToMetres function definition
// converts feets in array to metres
void feetToMetres(double* ptrv)
{
       for (int j = 0; j < MAX; j++)
       {
              *ptrv = *ptrv * 0.3048;
              ptrv++;
              // or just *ptrv++ *= 0.3048; instead of two separate operators
              // or just ptrv[j] = ptrv[j] * 0.3048;
       }
}
```

Example 13:

```
// Example 13
// passing array as argument to function (by pointer)
#include <iostream>
using namespace std;

void feetToMetres(double*, int);        // feetToMetres function prototype

int main()
{
       const int MAX = 5;               // element count in array as local variable => here
                                        // available only in main function
       double varray [MAX] = { 15.0, 34.6, 75.9, 23.7, 55.1 };

       feetToMetres(varray, MAX);        // covert all array elements from feet to metres

       // output the result
       for (int j = 0; j < MAX; j++)
              cout << "varray [ " << j << " ] = " << varray [ j ] << " metres" << endl;

       cout << endl;
       return 0;
}
```

```
//-------------------------------------------------------
// feetToMetres function definition
// converts feets in array to meters
void feetToMetres(double* ptrv, int n)
{
	for (int j = 0; j < n; j++)
	{
		*ptrv = *ptrv * 0.3048;
		ptrv++;
		// or just *ptrv++ *= 0.3048; instead of two separate operators
		// or just ptrv[j] = ptrv[j] * 0.3048;
	}
}
```

- **Passing *C-string* to a function**

Example 14:

```
// Example 14
// passing C-string as argument to function (by pointer)
#include <iostream>
using namespace std;

short int spaceCount(char *);		// spaceCount function prototype

int main()
{
	char str[255];
	//char *str = new char[255];
	cout << "Enter string > " ;
	cin.getline(str,255);
	int n = spaceCount(str);		// value returned from spaceCount function
						// is assigned to variable n
	cout << "Count of spaces is " << n << endl << endl;

	return 0;
}

//---------------------------------------------------------------
// spaceCount function definition
// counts spaces in C-string
short int spaceCount(char *s)
{
	short int count = 0;
	for (int i = 0; i < strlen(s); i++)
		if (s[i] == ' ')
			count++;
	return count;
}
```

- **Passing *string* class string to a function**

Example 15:

```
// Example 15
// passing string value as argument to function
#include <iostream>
#include <string>
using namespace std;

int spaceCount(string);                 // spaceCount function prototype

int main()
{
       string str;
       cout << "Enter string > " ;
       getline(cin,str);
       int n = spaceCount(str);         // value returned from spaceCount function
                                        // is assigned to variable n
       cout << "Count of spaces is " << n << endl << endl;

       return 0;
}

//-------------------------------------------------------
// spaceCount function definition
// counts spaces in a string
int spaceCount(string s)
{
       int count = 0;
       for (int i = 0; i < s.length(); i++)
              if (s[i] == ' ')
                     count++;
       return count;
}
```

Example 16:

```
// Example 16
// passing string value as argument to function (by value)
#include <iostream>
#include <string>
using namespace std;

void stringChange_test(string);         // stringChange_test function prototype

int main()
{
       string str;
       str = "string in functions";
       stringChange_test(str);          // str remains unchanged
       cout << str;                     // "string in functions"
       cout << endl << endl;

       return 0;
}

//---------------------------------------------------
// stringChange_test function definition
// changes the string
void stringChange_test(string s)
{
       s = "Changed";
}
```

Example 17:

```
// Example 17
// passing string value as argument to function (by reference)
#include <iostream>
#include <string>
using namespace std;

void stringChange_test(string&);        // stringChange_test function prototype

int main()
{
       string str;
       str = "string in functions";
       stringChange_test(str);          // str is changed by stringChange_test function
       cout << str;                     // "Changed"
       cout << endl << endl;

       return 0;
}

//---------------------------------------------------
// stringChange_test function definition
// changes the string
void stringChange_test(string& s)
{
       s = "Changed";
}
```

- **Parameters with *const* modifier**

Example 18:

```
// Example 18
// using constant parameters in function
#include <iostream>
using namespace std;

void constChange_test(const int&);      // constChange_test function prototype

int main()
{
       int a = 10;
       constChange_test(a);
       cout << a;

       return 0;
}

//--------------------------------
// constChange_test function definition
void constChange_test(const int& fa)
{
       fa = 15;            // compilation error
}
```

- **Passing function name to a function**

Example 19:

```
// Example 19
// passing function pointer as argument to functions
#include <iostream>
#include <time.h>
using namespace std;

typedef int (*PF) (int, int);           // new type named PF as pointer to function with
                                        // two parameters of type int and return value of
                                        // type int

void generate_array(int*,int);          // generate_array function prototype
void display_array(int*,int);           // display_array function prototype
void sort(int*,int,PF);                 // sort function prototype
//void sort (int*, int, int (*) (int,int));   // sort function prototype if no typedef
int compare1(int,int);                          // compare1 function prototype
int compare2(int,int);                          // compare2 function prototype
int compare3(int,int);                          // compare3 function prototype
int digitSum(int);                      // digitSum function prototype

int main()
{
       int n;
       cout << "Enter element count in array: ";
       cin >> n;
       int *arr = new int[n];           // dynamic array of n integer numbers

       generate_array(arr,n);           // generate_array function call
       display_array(arr,n);            // display_array function call
       sort(arr,n,compare1);            // sort function call
       cout <<"\n----- array sorted in ascending order -----\n";
       display_array(arr,n);            // display_array function call
       sort(arr,n,compare2);            // sort function call
       cout <<"\n----- array sorted in descending order -----\n";
       display_array(arr,n);            // display_array function call
       sort(arr,n,compare3);            // sort function call
       cout <<"\n----- array sorted in ascending order by digit sum in elements -----\n";
       display_array(arr,n);            // display_array function call

       delete [] arr;
       cout << endl;

       return 0;
}

//-------------------------------------------------------
// generate_array function definition
// generates array
void generate_array(int* mas, int size)
{
       srand(time(NULL));
       for (int i = 0; i < size; i++)
              mas[i] = rand()%1000;
              //*(mas+i) = rand()%1000;

}

//-------------------------------------------------------
// display_array function definition
// outputs array
void display_array(int* mas, int size)
{
       for (int i = 0; i < size; i++)
              cout << mas[i] << " ";
              //cout << *(mas+i) << " ";
}
```

```
//-------------------------------------------------------
// sort function definition
// bubble sort
void sort(int *a, int n, PF f)
//void sort(int *a, int n, int (*f) (int,int))       // sort function header if no typedef
{
	for (int i = 1; i < n; i++)
		for (int j = n - 1; j >= i; j--)
			if(f(a[j],a[j-1]) == -1)
			{
				int temp = a[j];
				a[j] = a[j-1];
				a[j-1] = temp;
			}
}

//-------------------------------------------------------
// compare1 function definition
// ascending order
int compare1(int a, int b)
{
	if(a < b) return -1;
	else return 1;
}

//-------------------------------------------------------
// compare2 function definition
// descending order
int compare2(int a, int b)
{
	if(a > b) return -1;
	else return 1;
}

//-------------------------------------------------------
// compare3 function definition
// by sum of digits in number, ascending order
int compare3(int a, int b)
{
	if(digitSum(a) < digitSum(b)) return -1;
	else return 1;
}

//-------------------------------------------------------
// digitSum function definition
// defines the sum of digits in number
int digitSum(int a)
{
	int sum = 0;
	while (a)
	{
		sum += a % 10;
		a /= 10;
	}
	return sum;
}
```

- **Parameters with default values**

Example 20:

```
// Example 20
// using parameters with default values
#include <iostream>
using namespace std;

int sum (int, int, int = 0);            // sum function prototype
                                        // third parameter has default value 0

int main()
{
       int x1, x2, x3;
       x1 = 10;
       x2 = 15;
       x3 = 5;
       cout << sum(x1, x2, x3) << endl;
       cout << sum(x1, x2) << endl;     //third argument is omitted
       cout << endl << endl;

       return 0;
}

//-------------------------------------------------------
// sum function definition
// calculates sum of 3 numbers
int sum (int a, int b, int c)
{
       return (a + b + c);              // if c value is not specified in sum
                                        // function call, then 0 value is used
                                        // for variable c (value by default,
                                        // defined in sum function prototype)
}
```

- **Function overloading**

Example 21:

```
// Example 21
// function overloading
#include <iostream>
using namespace std;

                                        // function max prototypes:
int max (int, int);                     // max between two integer numbers
float max (float, float);               // max between two real numbers
char max (char, char);                  // max between two symbols
char* max (char*, char*);               // max between two C-strings

int main()
{
       int a, b;
       cout << "Enter 2 integer values: ";
       cin >> a >> b;
       cout << "max(int,int) is "<< max(a,b) << endl;
       float x,y;
       cout << "Enter 2 real values: ";
       cin >> x >> y;
       cout << "max(float,float) is " << max(x,y) << endl;
       char c1, c2;
       cout << "Enter 2 symbols: ";
       cin >> c1 >> c2;
       cout << "max(char,char) is " << max(c1,c2) << endl;
```

```
	char s1[10], s2[10];
	cout << "Enter 2 strings (max 9 symbols each): ";
	cin >> s1 >> s2;
	cout << "max(string,string) is " << max(s1,s2) << endl;

	cout << endl << endl;

	return 0;
}

//-------------------------------------------------------
// max function definitions (overloaded versions)
// defines and returns maximum value
int max(int a, int b)
{
	if (a > b) return a;
	else return b;
}

float max(float a, float b)
{
	if (a > b) return a;
	else return b;
}

char max(char a, char b)
{
	if (a > b) return a;
	else return b;
}

char* max(char* a, char* b)
{
	if (strcmp(a,b) > 0) return a;
	else return b;
}
```

- **Function templates**

Example 22_1:

```
// Example 22_1
// using function templates
#include <iostream>
using namespace std;

// maxT function template definition
template <class Type>
Type maxT(Type a, Type b)
{
	if (a > b) return a;
	else return b;
}

int main()
{
	int a, b;
	cout << "Enter 2 integer values: ";
	cin >> a >> b;
	cout << "maxT() is " << maxT(a,b) << endl;
	float x,y;
	cout << "Enter 2 real values: ";
	cin >> x >> y;
	cout << "maxT() is " << maxT(x,y) << endl;
```

```
        char c1, c2;
        cout << "Enter 2 symbols: ";
        cin >> c1 >> c2;

        cout << "maxT() is " << maxT(c1,c2) << endl;
        char s1[10], s2[10];
        cout << "Enter 2 strings (max 9 symbols each): ";
        cin >> s1 >> s2;
        cout << "maxT() is " << maxT(s1,s2) << endl;  // will not work correctly
                                                      // because operation '>' is
                                                      // not defined for char arrays (for
                                                      // C-strings).
                                                      // To compare two C-strings strcmp
                                                      // function should be used.
                                                      // With strings like objects of
                                                      // of class string it would work
                                                      // correctly, because string object
                                                      // can be compared with other string
                                                      // object using '>' operation.
        cout << endl << endl;

        return 0;
}
```

Example 22_2 (with function prototype):

```
// Example 22_2
// using function templates
#include <iostream>
using namespace std;

template <class Type>
Type maxT(Type a, Type b);              // maxT function prototype

int main()
{
        int a, b;
        cout << "Enter 2 integer values: ";
        cin >> a >> b;
        cout << "maxT() is " << maxT(a,b) << endl;
        float x,y;
        cout << "Enter 2 real values: ";
        cin >> x >> y;
        cout << "maxT() is " << maxT(x,y) << endl;
        char c1, c2;
        cout << "Enter 2 symbols: ";
        cin >> c1 >> c2;

        cout << "maxT() is " << maxT(c1, c2) << endl;
        char s1[10], s2[10];
        cout << "Enter 2 strings (max 9 symbols each): ";
        cin >> s1 >> s2;
        cout << "maxT() is " << maxT(s1, s2) << endl; // will not work correctly, see
                                                      // comments in example 22_1

        cout << endl << endl;

        return 0;
}

// maxT function template definition
template <class Type>
Type maxT(Type a, Type b)
{
        if (a > b) return a;
        else return b;
}
```

Example 23_1:

```
// Example 23_1
// using function templates
#include <iostream>
using namespace std;

// find function template definition
// returns index of element if found or -1 if value not found
template <class atype>
int find(atype* arr, int size, atype value)
{
	for (int j = 0; j < size; j++)
		if( arr[j] == value)
			return j;
	return -1;
}

int main()
{
	const int n = 6;                                        // arrays' size
	char chrArr[] =   {'a', 'b', 'c', 'd', 'e', 'f'};       // array
	//char chrArr[] = "abcdef";                             // the same as previous
	char chValue = 'c';                                     // value to search

	int intArr[] =    {1, 3, 5, 9, 11, 13};                 // array
	int intValue = 7;                                       // value to search

	double dblArr[] = {1.0, 3.0, 5.0, 9.0, 11.0, 13.0};     // array
	double dblValue = 4.0;                                  // value to search

	cout << "\n'c'in chrArray: index = " << find(chrArr, n, chValue);
	cout << "\n 7 in intArray: index = " << find(intArr, n, intValue);
	cout << "\n 4 in dblArray: index = " << find(dblArr, n, dblValue);

	cout << endl << endl;
	return 0;
}
```

Example 23_2 (with function prototype)**:**

```
// Example 23_2
// using function templates
#include <iostream>
using namespace std;

template <class atype>
int find(atype* arr, int size, atype value);        // find function prototype

int main()
{
	const int n = 6;                                        // arrays’ size
	char chrArr[] =   {'a', 'b', 'c', 'd', 'e', 'f'};       // array
	//char chrArr[] = "abcdef";                             // the same as previous
	char chValue = 'c';                                     // value to search

	int intArr[] =    {1, 3, 5, 9, 11, 13};                 // array
	int intValue = 7;                                       // value to search

	double dblArr[] = {1.0, 3.0, 5.0, 9.0, 11.0, 13.0};     // array
	double dblValue = 4.0;                                  // value to search

	cout << "\n'c'in chrArray: index = " << find(chrArr, n, chValue);
	cout << "\n 7 in intArray: index = " << find(intArr, n, intValue);
	cout << "\n 4 in dblArray: index = " << find(dblArr, n, dblValue);

	cout << endl << endl;
	return 0;
}
```

```
// find function template definition
// returns index of element if found or -1 if value not found
template <class atype>
int find(atype* arr, int size, atype value)
{
	for (int j = 0; j < size; j++)
		if( arr[j] == value)
			return j;
	return -1;
}
```

- **Recursive functions**

Example 24:

```
// Example 24
// recursion
#include <iostream>
using namespace std;

int fact(int);		// fact function prototype

int main()
{
	int a;
	cin >> a;
	cout << "factorial = " << fact(a);

	cout << endl << endl;
	return 0;
}

//-------------------------------------------------------
// fact function definition
// recursive function, calculates factorial of a specified number
int fact(int a)
{
	if (a == 1) return 1;
	else return fact(a-1) * a;
}
```

- ***main* function with parameters**

Example 25:

```
// Example 25
// main function with parameters
#include <iostream>
using namespace std;

int main(int argc, char* argv[])
{
	for (int i = 0; i < argc; i++)
		cout << argv[i] << endl;
	if (argc == 1)
		cout << "There are no arguments\n";
	else
	{
		int sum = 0;
		for (int i = 1; i < argc; i++)
			sum += atoi(argv[i]);
		cout << "Sum of arguments is " << sum << endl;
	}

	cout << endl;
	return 0;
}
```